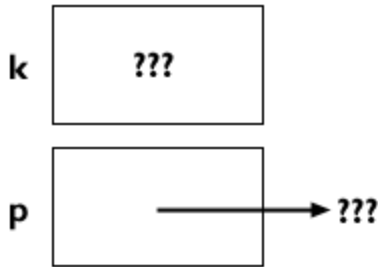


Topic 1.3

1.3: Pointers

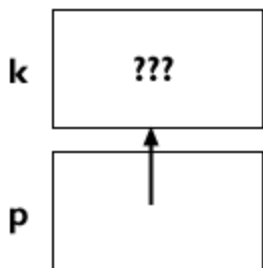
Pointers are rampant in C programs. We can declare a pointer to an integer variable as follows (remember that uninitialized automatic variables have an undefined or "garbage" value):

```
int k; /* k is an int variable. */  
int *p; /* p is a pointer to an int. It initially points to an unknown memory location. */
```



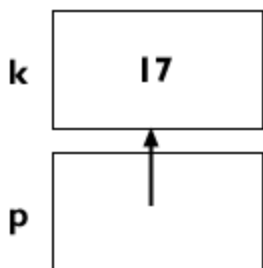
The ampersand `&` is used to take the address of a variable:

```
p = &k; /* p now points at k. */
```



Note that *p* contains a value that is interpreted as an address. If *k* resides at memory location 4356, the value of *p* is now 4356. We can now use the pointer to modify the value of *k*:

```
*p = 17; /* Store 17 where p points, namely k. */
```

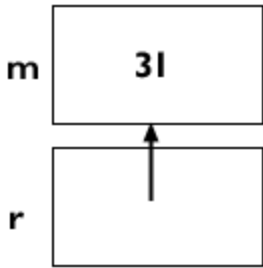


The asterisk `*` is the *dereference* operator. The notation `*p`, where *p* is a pointer, means "the thing *p* points to."

Pointers, as well as other variables, can be initialized in their declaration.

```
int m=31;

int *r=&m; /* r points to m, which contains
           the value 31. */
```



Be careful when using pointers in `scanf` or `printf` invocations:

```
printf("The value where r points is %d\n", *r);scanf("%d", r );
// NOTICE: no &.
```

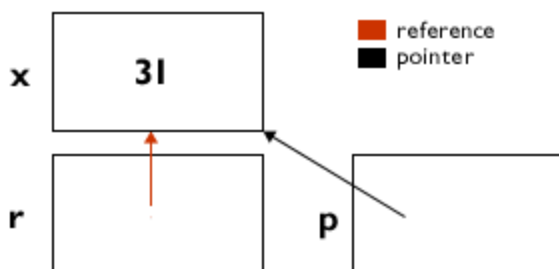
References

In addition to pointers, C++ has *references*. The following example illustrates the concept:

```
int x;
int &r=x; // r is a reference to x.
int *p=&r;

*p = 30;

++r
```



When declaring a reference, you **must** assign it a target, that is, something that it references. Do **not** use the address-of operator `&` for this initialization. The compiler automatically computes a reference to the designated target.

All uses of the reference actually refer to the target. The reference is effectively an alias, or another name, for the target. It is impossible to compute the actual address of the reference itself. Whenever the address-of operator is applied to a reference, it yields the address of the reference's target.

In practice, the bit pattern used for a reference is the same as that of a pointer to the object. However, the implementation of "reference" itself is not directly available to programmers. The compiler may implement a reference as it sees fit. For example, a reference *could* be a pointer with all the bits complemented, then reversed.

Topic 1.4 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-1-dot-4>)